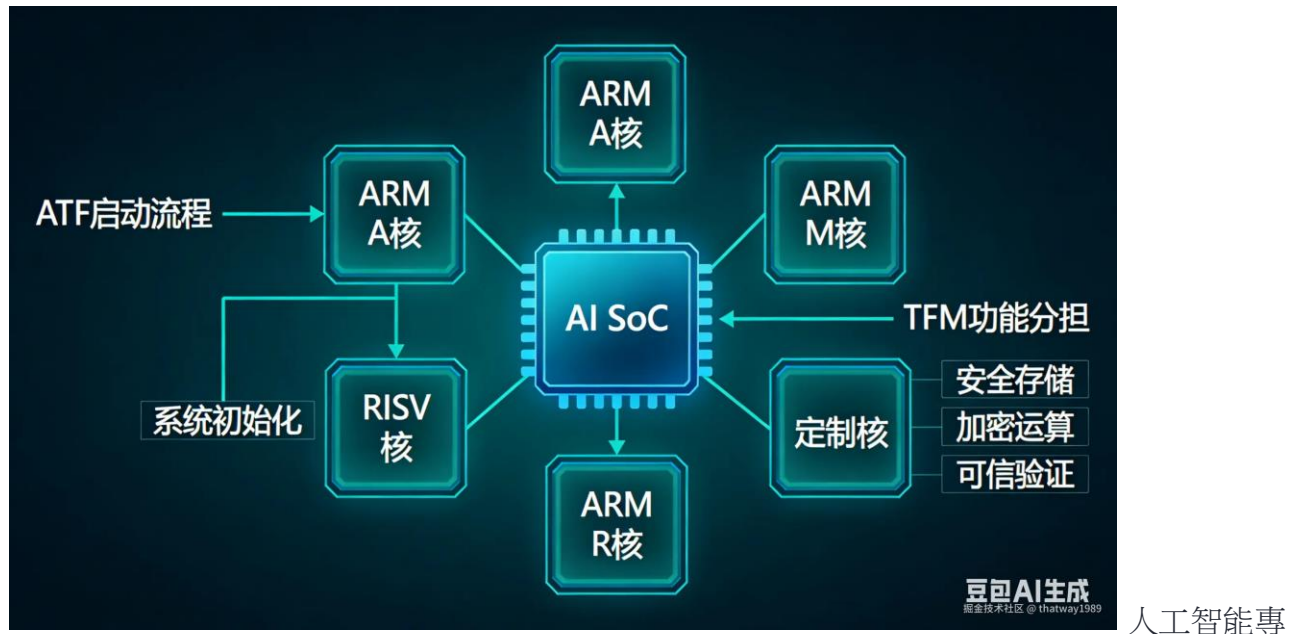


ARM TFM-1 介紹及代碼下載、運行與适配指南

2026-03-3170 閱讀 15 分鐘



用晶片通常會整合多個核心，包括 ARM 系列的 A 核心、M 核心、R 核心，以及 RISCV 核心和自訂核心等。之前已經多次提到 ATF 安全啟動的過程：都是以 A 核心作為起點，再進行安全啟動。

這裡有一個問題：是選擇 A 核安全，還是 M/R 核安全呢？答案顯而易見：M/R 核更安全。

那麼，下一個問題是：既然 M 核心更安全，那為何不把安全啟動和讀取密鑰這些系統中最重要任務交給 M 核心呢？想到這裡，本文的主角 TFM 就登場了。TFM 同樣是 ARM 的開源軟體，可在多核 SoC 中與 A 核心中的 ATF 協同工作，共同承擔核心安全功能。如此一來，一切都完美了！

純使用 ATF 通常是手機的選擇，因為手機上只有 A 核心，沒有 M 核心、R 核心等。不過，對於有安全需求的人工智能 SoC 及其他多核心 SoC 而言，使用 M 核心會更適合。

1. TFM 與 ATF 的對比

1.1 為何 M 核比 A 核更安全？



在嵌入式/車載/安全芯片領域，有一個非常經典的結論：M 核心天生就比 A 核心更難被攻破，因此更適合用來作為安全根（RoT）。

原因並非是 M 核“性能更強”，而是其架構簡單、攻擊面極小、具有天然的隔離功能，且權限管理嚴格。

M 核與 A 核安全特性對比表

對比維度	Cortex-M 核心（M 核心）	Cortex-A 核心（A 核心）
M 核是「微控制器」，A 核是「應用處理器」	- 在裸機/RTOS 上運行 - 代碼量少、啟動快、邏輯簡單 - 几乎沒有複雜的 MMU、異常級別、多任務調度	- 運行 Linux / Android 系統 - 擁有完整的操作系統、驅動程式、檔案系統、網路功能及各種應用程式 - 代碼量巨大，漏洞自然也就更多了
A 的權限層級眾多，因此容易成為攻擊的目標	- 只有普通線程模式、處理異常模式 - 配合 TrustZone for M（安全/非安全模式）使用 - 權限模型極其簡單，幾乎沒有“提權路徑”	- 具有完整的異常等級划分：EL0（應用層）→ EL1（核心層）→ EL2（管理程序層）→ EL3（安全監控層） - 攻擊者只要找到一個內核漏洞，就能：提升權限 → 讀寫任意內存 → 劫持控制流 → 攻破 TEE

對比維度	Cortex-M 核心（M 核心）	Cortex-A 核心（A 核心）
M 核心沒有 MMU，也沒有虛擬內存，因此更易於控制。	- 仅使用 MPU（内存保护单元）：規則簡單、配置靜態 - 沒有複雜的頁表、沒有缺頁 - 無法透過竄改頁表來越權訪問 - 硬體行為可預測，攻擊難度陡增	- 透過 MMU 產生虛擬地址，從而產生：頁表、缺頁異常、核心態/使用者態切換、以及大量複雜的邏輯處理 - 每一個都是潛在漏洞點
M 核通常不用于运行大型操作系统，因此其被攻擊的機會極低。	- 只提供安全服務：加密、驗簽、安全儲存、隨機數 - 代碼量少、功能單一 - 不需要上網、也不需運行複雜的應用程式 - 外設訪問嚴格受限	- Linux：數百萬行代碼的驅動程式、網路協議堆疊、檔案系統、各種用戶端程式，以及大量的外設驅動程式 - 天然就是漏洞重災區
M 核可以在物理/硬件上完全隔離	- 在真正的 SoC 中：M 核可以獨佔一部分 SRAM（安全內存） - TZASC 及外設安全配置可實現：A 核與 M 核安全區完全不可見、無法被訪問	- A 核的安全世界（TEE）與非安全世界（Rich OS）共享 CPU、緩存和總線 - 側信道、時序攻擊更容易滲透
M 核執行環境已固化，不易被篡改	- M 固件通常儲存在 OTP、ROM 或加密 Flash 中 - 啟動流程簡短、信任鏈也較短 - 一旦配置好安全策略，就很難再進行動態修改了	- 核心可加載模块、驅動程式可動態載入、設備樹可被竄改 - 流程冗長、環節眾多，任何一個環節出現問題，整個流程就會崩潰
真正關鍵：M 核心是為了“可靠、控制、安全”而設計的	- 設計目標：即時、可靠、低功耗、控制安全 - 硬件隔離原生支持 - 安全是與生俱來的特質，而非後天補充的東西	- 設計目標：高效率、多任務處理、能夠運行複雜的系統、兼容各種軟體 - 安全是附加功能（TrustZone、TEE）

A 核是用來“處理各種任務、運行系統”的，本質上很複雜，且存在許多漏洞；**M 核**則是用來“保護系統、維護安全”的，本質上很簡單，具有出色的隔離功能，受攻擊的機會極少。

因此，正確的做法是：將安全根（RoT）置於 M 核（TF-M）中，而 A 核（ATF+TEE）則僅用於輔助隔離功能。

1.2 TFM 與 ATF 的搭配使用



在 A+M 核心 SoC 中，ATF（TF-A）負責 A 核心側的 EL3 安全監控與世界切換，TF-M 則負責 M 核心側的 SPE 安全服務與隔離。二者透過硬件隔離、安全啟動機制以及標準化的 IPC/SMC 通訊協議，能夠將高風險的 A 核心與安全資產徹底隔離，因此這是一種非常安全的架構方案。

1. ATF（Trusted Firmware-A / TF-A）：A 核心的安全基石

運行環境：A 核心（Cortex-A）的 EL3 最高特權級，僅安全模式。

核心職責：

- 構建完整安全啟動信任鏈（BL1→BL2→BL31→BL32）。
- 管理安全/非安全世界切換（SMC 異常捕獲、路由、返回）。
- 提供 PSCI 電源管理、安全中斷、系統監控等底層服務。
- 將 A 核心側的 TEE 操作系統（如 OP-TEE）與 Rich 操作系統（Linux/Android）分開。

安全價值：守住 A 核的最高權限入口，防止非安全側越權訪問安全資源。

2. TF-M（Trusted Firmware-M）：M 核心的安全堡壘

運行環境：M 核心（Cortex-M33/M55 等）的 Secure Processing Environment（SPE），具有硬件隔離功能。

核心職責：

- 實現 PSA RoT（平台安全根）：安全啟動、加密、安全儲存（ITS/PS）、固件更新、平台認證。
- 管理 SPE/NSPE 的隔離功能（TrustZone 或物理核心隔離），並提供標準化的 PSA 安全 API。
- 托管密鑰、憑證、敏感數據，僅透過受控接口提供外部服務。
- 安全價值：將最敏感的安全資產與服務置於遭受攻擊的風險最小的 M 核心 SPE 中的受信任固件中。

1.2.1 安全協同方案（A 核遭受攻擊時如何保護系統）

方案總覽：雙信任根 + 硬件強隔離 + 跨核安全通訊

- M 核（TF-M）：作為主要根憑證，負責托管所有的核心安全服務與密鑰。
- A 核（ATF+TEE）：作為 Secondary RoT，負責 A 核側的安全監控與隔離。
- 關鍵原則：A 核永遠不能直接訪問 M 核 SPE 的內存與資源，所有交互必須透過標準化、可審計的安全通道 Trusted Firmware 來進行。

安全啟動鏈協同（從上電就建立信任）

BootROM（不可變，信任根）

↓ 驗證並加載

SCFW（可選，M 核系統控制器，配置安全策略）

↓ 驗證並加載

ATF BL1 → BL2 → BL31（EL3）

↓ BL31 驗證並加載

A 核 TEE OS（BL32，如 OP-TEE）

↓ 同時/按序

M 核 TF-M（SPE）→ 驗證並啟動 M 核 NSPE

安全要點：

- ATF 與 TF-M 的鏡像都經過 BootROM/SCFW 的簽名驗證，任何篡改都會導致無法啟動。
- M 核 TF-M 應優先啟動，或與 A 核 ATF 同時啟動，以便盡早建立安全基礎，防止在 A 核尚未準備就緒時遭受攻擊。
- OTP 儲存了 ROTPK（根公鑰），作為整個系統信任鏈的起點。

1.2.1 硬件隔離配置（最核心的安全屏障）

- 物理核心隔離：
 - M 核心專用於運行 TF-M（SPE），A 核心則用於運行 Rich OS 與 TEE Trusted Firmware。
 - 透過 TZASC（TrustZone Address Space Controller）與 MPU，徹底劃分 A/M 核心的內存、外設訪問權限。
 - A 核心沒有任何權限，無法直接讀寫 M 核心 SPE 的內存與寄存器，以及 Trusted Firmware。
- 中斷與 DMA 隔離：
 - 安全中斷僅會被路由到 M 核心的 TF-M 或 A 核心的 EL3，非安全側無法攔截或竄改。
 - DMA 控制器被配置為僅允許非安全側存取指定的內存區域，從而防止 DMA 攻擊竊取安全數據。

1.2.1 運行時通訊

跨核安全通訊（唯一受控交互方式）

（1）A 核 → M 核（請求安全服務）

1. A 核非安全側（Linux）→ 通過 TEE Client API → A 核 TEE OS（OP-TEE）。
2. TEE OS → 啟動 SMC 調用 → A 核 EL3（ATF BL31）。
3. ATF → 驗證請求的合法性 → 通過硬件 mailbox/IPC 控制器 → 發送到 M 核。
4. M 核 TF-M → 接收、鑒權、執行安全服務（加密/存儲/驗簽）→ 返回結果。
5. 結果沿原路徑返回，全程加密與審計。

（2）M 核 → A 核（安全事件通知）

- M 核 TF-M → 透過安全中斷或郵件匣 → 通知 A 核 ATF/TEE。
- 僅用於安全告警、狀態同步、固件更新觸發等，不傳輸敏感數據。

通訊安全規則：

- 最小權限接口：僅暴露必要的 PSA 安全 API，不開放底層寄存器/內存訪問。
- 請求鑒權：對每個跨核請求進行身份驗證與權限檢查。
- 數據加密：交互數據會使用會話密鑰進行加密，以防止竊聽與篡改。

- 審計日誌：所有安全操作都會被記錄在安全存儲系統中，且不可被篡改。

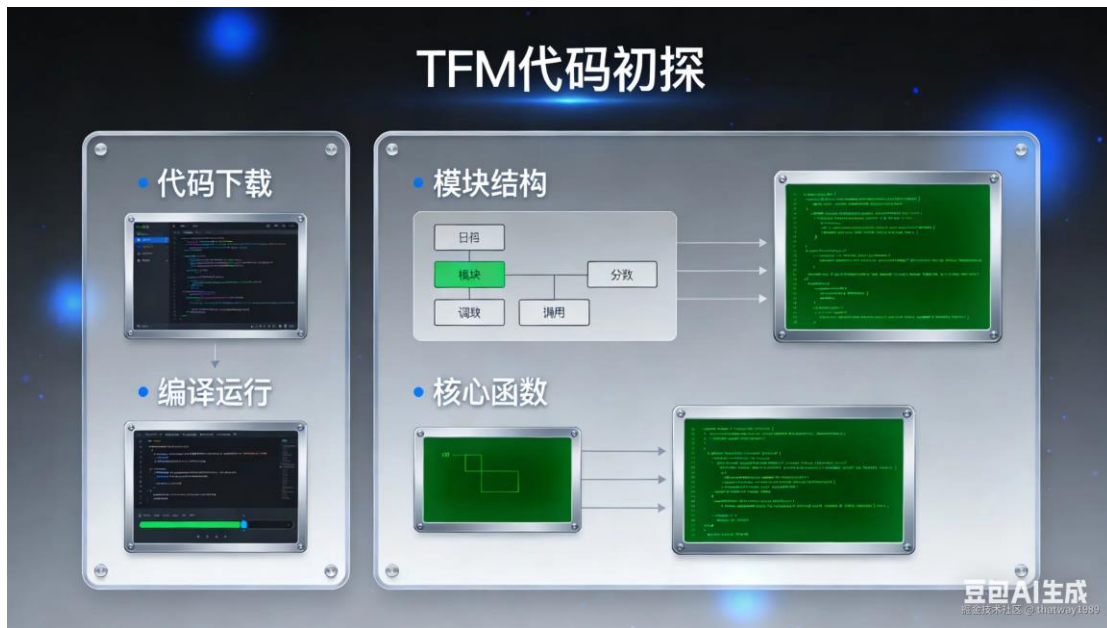
1.2.3 安全服務部署策略（降低 A 核風險）

- 必須放在 M 核 TF-M 的服務中（A 核絕對不能觸碰）：
 - 根密鑰、設備唯一密鑰、憑證儲存（ITS/PS）。
 - 安全啟動驗簽、固件更新驗簽、平台完整性認證。
 - 硬件隨機數生成、核心加解密算法（AES/RSA/ECC）。
- 可放在 A 核 TEE 的服務（降低 M 核負載）：
 - 與業務相關的安全邏輯（如支付、DRM）。
 - 需要 A 核心側快速回應的安全計算。
- A 核非安全側（Linux）：僅負責處理業務邏輯，不保存任何長期密鑰。所有安全操作均通過 TEE→TF-M 鏈路來執行。

1.2.4 異常與攻擊應對（A 核淪陷後如何止損）

- 隔離熔斷：當 M 核 TF-M 檢測到 A 核出現異常情況（例如頻繁的非法請求或數據篡改時），會立即切斷跨核通訊，並凍結 A 核使用安全服務的權限。
- 安全狀態回滾：當 A 核被攻破後，M 核 TF-M 可觸發安全恢復流程，重新驗證並載入 A 核固件，此過程不會影響安全根。
- 最小暴露原則：即使 A 核完全失控，M 核 SPE 的密鑰與服務依然安全，系統核心資產不會泄露。

2. TFM 代碼初探



有了上述介紹之後，你在選擇方案時應該已經有個大概的了解了。接下來，我們來實際操作一下代碼吧。至於閱讀代碼的部分，可以讓 AI 來輔助。

2.1 代碼下載

```
git clone https://git.trustedfirmware.org/TF-M/trusted-firmware-m.git/
```

在 cmake 編譯時，也會自動下載 mbedtls 等所需的依附套件。

2.2 代碼編譯

1. 編譯工具安裝：

arm-none-eabi (GNU Arm Embedded Toolchain 10-2020-q4-major) 10.2.1 20201103 (正式版)

官網：

<https://developer.arm.com/tools-and-software/open-source-software/developer-tools/gnu-toolchain/gnu-rm/downloads>

如果本地有編譯工具，修改下面代碼指定即可：

```
--- a/config/config_base.cmake
+++ b/config/config_base.cmake
@@ -12,7 +12,7 @@
```

```
set(TFM_TOOLCHAIN_FILE           ${CMAKE_SOURCE_DIR}/toolchain_GNUARM.cmake CACHE
FILEPATH      "Path to TFM compiler toolchain file")
set(TFM_PLATFORM                 ""          CACHE STRING      "Platform to build TF-M
for. Must be either a relative path from [TF-M]/platform/ext/target, or an absolute
path.")
-set(CROSS_COMPILE                arm-none-eabi CACHE STRING    "Cross-compilation
triplet")
+set(CROSS_COMPILE                /XXX/arm-gnu-toolchain-14.3.rel1-x86_64-arm-none-
eabi/bin/arm-none-eabi CACHE STRING    "Cross-compilation triplet")
```

2. 在編譯之前，請先確認 Python 3 是否缺少 **Ninja** 這個套件。如果缺少，請使用 **pip** 進行安裝：

```
pip3 install ninja
```

3. 接著進入 TF-M 根目錄下進行 **cmake** 設定：

```
cmake -S . -B cmake_build -DTFM_PLATFORM=arm/mps2/an521 -
DTFM_TOOLCHAIN_FILE=toolchain_GNUARM.cmake -DTEST_NS=ON -DTEST_S=ON -
DTFM_PSA_API=ON -DBL2=OFF
```

參數說明：

TFM_PLATFORM：該宏用於指定所使用的平台。目前使用的是 **arm-mps2-an521** 平台，而該平台實際上模擬了 **cortex-m33** 處理器。

DTFM_TOOLCHAIN_FILE：使用編譯器類型，此處指定使用 **arm-none-eabi** 編譯工具鏈。

TEST_NS=ON：編譯 **none-secure** 測試套件。

TEST_S=ON：編譯 **secure** 測試套件。

DTFM_PSA_API=ON：使用编译后的 **PSA** 接口。

BL2=OFF：如果不使用 **BL2** 安全启动模式，使用 **qemu-system-arm** 来运行 **BL2** 的镜像时会出现异常。

注：如需了解更多編譯宏選項的詳細資訊，請參閱

docs/getting_started/tfm_build_instruction.rst

4. 進入到 **cmake_build** 文件夾：

執行 **make**

編譯成功後，會在 **cmake_build** 資料夾下生成 **bin** 資料夾。

arm studio：官網上：trustedfirmware-m.readthedocs.io/en/latest/g... 需要付費用戶才能使用。我找了一個可以自己嘗試的網站：gitcode.com/Premium-Res...

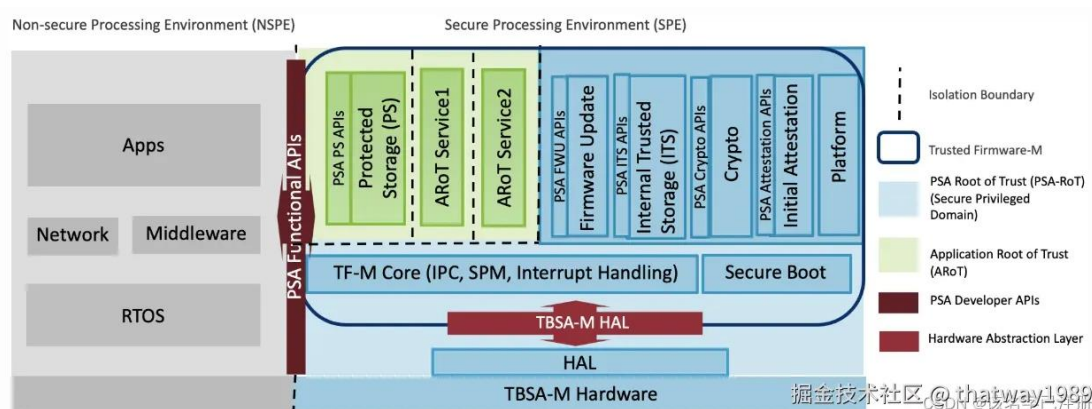
如果要學習的話，可以讓 Claude 或 OpenCLaw 去閱讀官方文檔：trustedfirmware-m.readthedocs.io/en/latest/g... 然後再結合代碼以及 ARM 官網 developer.arm.com/ 上的手冊來進一步理解。

同時，這些網上的資料也是 AI 在制定架構方案、進行代碼審核和測試等環節中所必需的技能，因此必須將其納入你的技能清單中。

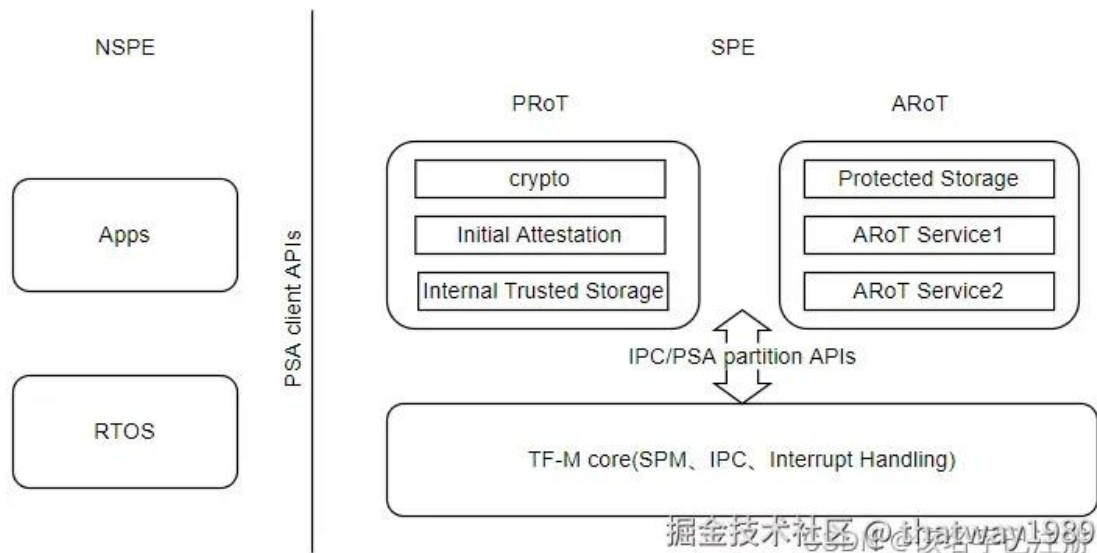
2.4 架構初探

參考：zhuanlan.zhihu.com/p/651683753

TF-M 是一種基於 PSA 架構開發的安全固件，本文所介紹的內容均基於 v1.5 版本。下圖為官方提供的圖片，展示了 TF-M 的各項詳細功能。



簡要的框圖如下：



其核心為 PSA 架構，以及 SPM 對分區線程、IPC 等機制的管理。各分區透過分區線程來運行，並提供一種或更多種服務。NSPE 負責運行 RTOS，而任務則在 RTOS 上執行。當任務需要使用與安全相關的服務時，可透過 PSA client APIs 來請求相關服務。SPM 會透過 IPC 機制通知對應的分區線程，使其執行該服務請求。服務完成後，結果會被返回給 client。

2.4 代碼初探

參考：blog.csdn.net/weixin_4308... 代碼入口：tfm/secure_fw/spm/cmsis_psa/main.c

Main()

```
->tfm_arch_set_msplim((uint32_t)&REGION_NAME(Image$$, ARM_LIB_STACK, $$ZI$$Base)); //
// 设置堆栈限制
->fih_delay_init();
->tfm_core_init()
->tfm_hal_set_up_static_boundaries
->configure_mpu(rnr++, base, limit, XN_EXEC_OK, AP_R0_PRIV_UNPRIV); //
// 配置各个内存边界的读写属性
->tfm_hal_platform_init() //核心函数，需要重点展开分析
->__enable_irq(); //使能中断
->stdio_init(); //初始化串口
-> corstone1000_watchdog_init() //初始化看门狗
-> fwu_metadata_init()
-> FWU_METADATA_FLASH_DEV.Initialize(NULL) //初始化 FWU
// metadata 所在的 FLASH，就是 32MB 那个
-> flash_info = FWU_METADATA_FLASH_DEV.GetInfo()
-> corstone1000_host_watchdog_handler_init()
```

```

->tfm_plat_otp_init()
->tfm_plat_provisioning_perform()
    -> tfm_plat_otp_read    //读 OTP 值
->tfm_arch_config_extensions()
    ->SCB->NSACR |= SCB_NSACR_CP10_Msk | SCB_NSACR_CP11_Msk;
->SCnSCB->CPPWR |= SCnSCB_CPPWR_SUS11_Msk | SCnSCB_CPPWR_SUS10_Msk;
    ->SPMLOG_DBGMSGVAL("TF-M isolation level is: ", TFM_LVL);
    ->SPMLOG_INFMSGC("\033[1;34m Booting TFM v"VERSION_FULLSTR"\033[0m\r\n"); /* Print the
TF-M version */
->tfm_arch_clear_fp_status();    //汇编代码
->tfm_core_handler_mode();    //发送 SVC 命令，触发 tfm_spm_init()，这个函数是 PSA 的重点

```

//tfm_core_handler_mode()函数的定義如下：它會調用 SVC 命令，並傳遞一個參數 TFM_SVC_SPM_INIT。最終，該函數會調用 SVC_Handler，再根據所給的 ID，調用 tfm_spm_init()函數。後續將詳細說明 tfm_spm_init()函數的具體功能。

```

__attribute__((naked)) void tfm_core_handler_mode(void)
{
    __ASM volatile("SVC %0          \n"
                   "BX LR          \n"
                   : : "I" (TFM_SVC_SPM_INIT));
}

```

PendSV 回應 PendSV（可懸起的系統調用）。它與 SVC 共同使用。SVC 異常必須立即得到回應，應用程式在執行 SVC 時，都希望自己的請求能立刻得到處理。而 PendSV 則不同，它可以像普通的中斷一樣被暫停或掛起。

PendSV_Handle

SVC 回應

SVC_Handler

```

->BL    tfm_core_svc_handler
    ->tfm_core_svc_handler
        -> case TFM_SVC_SPM_INIT:    tfm_spm_init()    //初始化 SPM
        -> case TFM_SVC_GET_BOOT_DATA:
tfm_core_get_boot_data_handler(svc_args);
        -> case TFM_SVC_PREPARE_DEPRIV_FLIH:
tfm_flih_prepare_depriv_flih (struct partition_t *)svc_args[0], (uintptr_t)svc_args[1]);

```

```

-> case TFM_SVC_FLIH_FUNC_RETURN:
tfm_flih_return_to_isr(svc_args[0], (struct context_flih_ret_t *)msp);
-> default: svc_args[0] = SVC_Handler_IPC(svc_number, svc_args,
exc_return); //根据 Client 端调用 SVC 命令时, 传入的 ID, 进入不同的 case 处理函数

```

tfm_spm_init

```

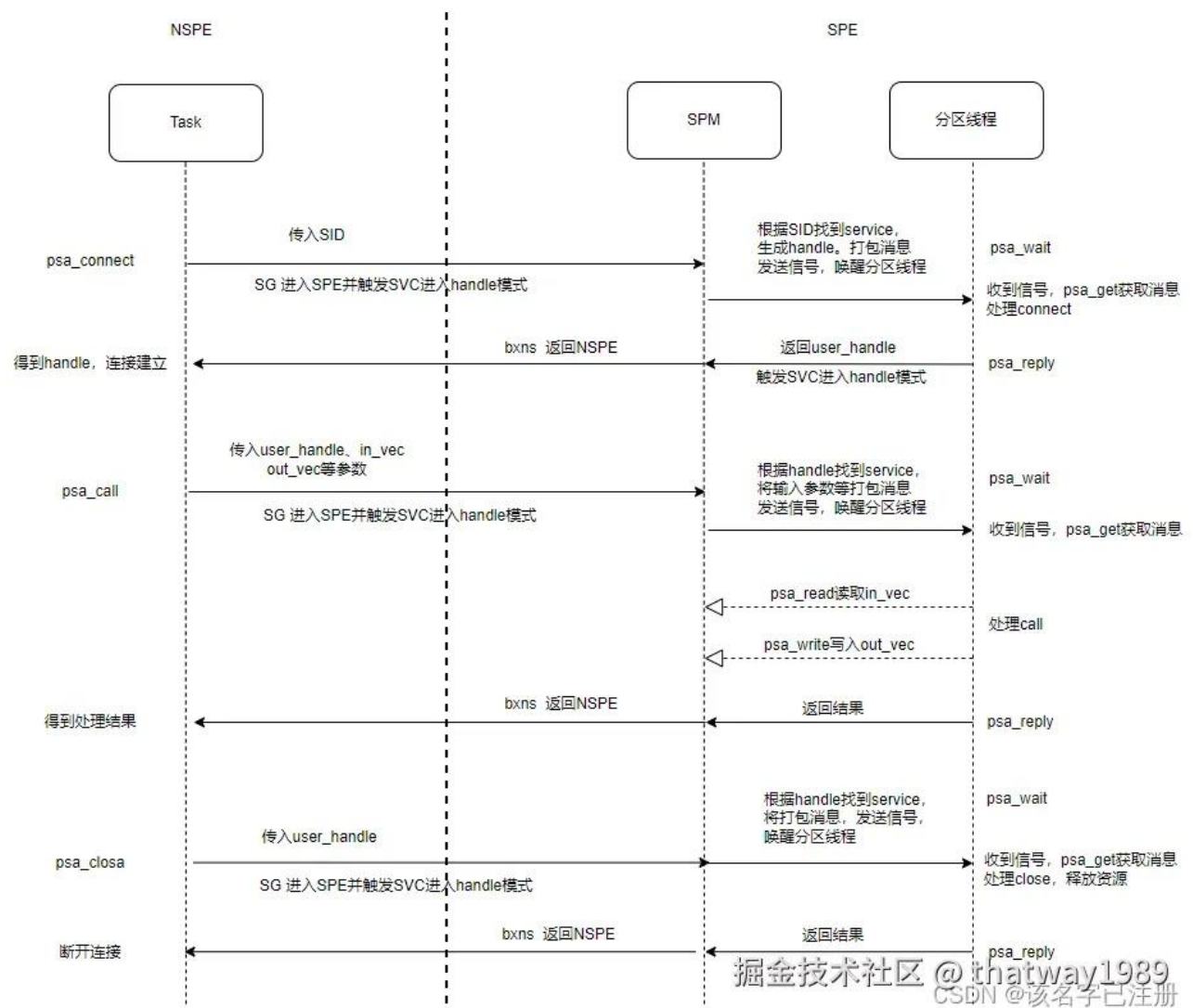
tfm_spm_init
->tfm_pool_init(conn_handle_pool, POOL_BUFFER_SIZE(conn_handle_pool), sizeof(struct
conn_handle_t),
CONFIG_TFM_CONN_HANDLE_MAX_NUM)
->load_a_partition_assuredly //从 section: .part.load 段中, 依次获取一个 一个的
partition_load_info_t
-> p_ptldinf->psa_ff_ver //检查 Magic 是否是: 0x5F5F0000
->UNI_LIST_INSERT_AFTER(head, partition, next) //加到链表头部
-> load_irqs_assuredly(partition) //从 partition 中, 获取到 struct
irq_load_info_t
->p_irq_info->init(p_partition, p_irq_info)
-> mailbox_irq_init() //初始化 mailbox 中断
-> tfm_hal_irq_enable(p_irq_info->source) //使能中断
-> backend_instance.comp_init_assuredly(partition, service_setting) //
backend_instance 定义在 backend_ipc.c 中
-> backend_instance.system_run() //开始调度

```

在 section:.part.load 段中, 定義了 tfm_ns_mailbox_agent_load 這個結構體。該結構體是**在 Cmake 檔案中, 透過解析 yaml 檔案後自動生成的**。

安全服務的處理過程依賴於 TF-M 所實現的 IPC 機制。這是一種以連接響應為基礎的服務器與用戶端模型。

從 psa_connect → psa_call → psa_close 的流程：



3. TFM 代碼移植與適配

TFM代码移植适配技术方案



如果我們要開發一個新的 SoC 項目，並且需要使用 TFM，但很有可能原生的 TFM 並不支持該芯片。因此，就需要對 TFM 進行移植和調整，這也是程序員的基本工作內容。主要有兩種處理方式：

1. 根據 CPU 類型，找一個相同或類似的型號，只要架構一致，編譯器就能夠進行編譯和運行。將其複製過來後，修改一下資料夾名稱和產品名稱，確保能夠正常編譯。之後再將其上板調試。首先以註釋的方式來測試整個流程，例如點燈、串口初始化等。接下來就是漫長的調試過程了。這種方法的優點是效率較高，缺點是代碼中包含了很多不必要的功能，不太純淨。因此，比較適合新手使用。
2. 另一種方法適合老手使用：直接新建一個空的產品資料夾，需要哪段代碼就自己新建檔案並編寫。這樣的代碼通常比較簡潔。如果編寫代碼的技術較高，出現的錯誤也會比較少。

3.1 軟硬件選型

1. 首先，需要根據業務需求來選擇合適的核型，例如 M3、M4、M5 等，以滿足性能與安全方面的要求。
2. 一旦核心部分確定了，那麼其指令集和架構也就確定了。接下來就需要選擇編譯器了，一般來說，開源項目都會使用 gcc。
3. 軟體代碼框架的選擇：例如若使用 TFM，則應選擇最近一個的文件基線版本。
4. 然後在 TFM 中尋找相同或相近的核的定製代碼。方法很簡單，只要全局搜索核的名稱即可，或者搜索 ARM 架構的名稱，例如 ARMV8。
5. 接下來就是將其編譯成第一版的 bin 檔案。

3.2 軟體移植

軟體的修改，首先就是編譯工具和編譯腳本的修改。這裡就不詳細說明了，待後續討論代碼的相關文章時再詳細說明。

3.2.2 bin 檔案組織

有了 bin 文件之後，TFM 的 bin 文件主要包含 BL1_1/BL1_2/BL2 這幾種。那麼，這些 bin 文件應該存放在 Flash 的哪個區域呢？又該如何安排 ATF 的 bin 文件，以及 SoC 中其他核心的 bin 文件呢？那些體積較大、或需要較晚才被載入的 bin 文件，則可以存放在 UFS 中。所有這些都需要進行統一的規劃與設計。

BL1_1 的作用顯然是存放 SoC 內部的 BootROM。BootROM 屬於可信的系統部件，因此不可被修改。那麼，為何要將 BL1 分成兩部分呢？原因是 BootROM 的存儲成本較高，且可用空間有限。因此，只需將與安全認證相關的資料存放在 BootROM 中即可。BL1_2 則用來存放 Flash 資料。首先，Flash 資料會被暫存到 SRAM 中，然後再執行認證流程。認證通過後，BL2 便會啟動其他核心的載入過程。

因此，對應於 Flash 的地址規劃必須清晰明確，並需進行統一設計。一旦地址確定下來，就可以在代碼中對相關地址進行修改了。

3.3.3 驱动程序修改

根據我們 SoC 上所使用的串口，修改相關的串口地址等配置。一旦串口能夠正常使用，就可以正式進行調試了，那麼就有希望了。

第二個比較重要的驅動就是 Flash 驅動。只要這個驅動正常運作，就能從 Flash 中讀取 bin 文件，進而載入並運行下一個 bin 文件了。

最後，一般的 SoC 中，所有核心的軟體都由同一個代碼庫管理。因此，TFM 的代碼需要單獨建立一個 Git 倉庫。接下來，必須修改系統的編譯腳本（如 buildroot 或其它 CMake 等編譯工具），將 TFM 的代碼整合進去。同時，還必須確保 TFM 與系統中的編譯器相容。盡量使用與其他系統相同的編譯器，這樣就能大幅簡化源碼的編譯環境配置。

需要注意的是，编译 TFM 时会下载一些库文件，例如 CMSIS 等。在集成时，可以将这些文件本地下载后直接推送到 git 仓库中，或者使用 Conan 包管理器来统一管理这些文件，从而便于后续的升级和隔离操作。

後記：

篇幅有限，只能簡單介紹一下 TFM 的來龍去脈。後續會有專文詳細說明相關代碼。另外，如果你想學習某個領域的知識，可以讓 AI 幫你撰寫學習資料。AI 無法解決那些你根本不知道的問題，而這正是拓展知識面的好機會。

運用 AI 最有效的方式，就是用於那些你知道但不懂的領域，這樣才能獲得最大效益。但對於那些你已經了解的領域，AI 的專業程度就比不上你了。至於那些你完全不知道的領域，就算 AI 再解釋，你也依然不懂。總之，就是：不懂的領域，AI 能幫助你理解；已懂的領域，AI 根本派不上用場；完全不知道的領域，AI 的解釋也是白費力氣。

“什麼都懂一點，什麼都不精通。”